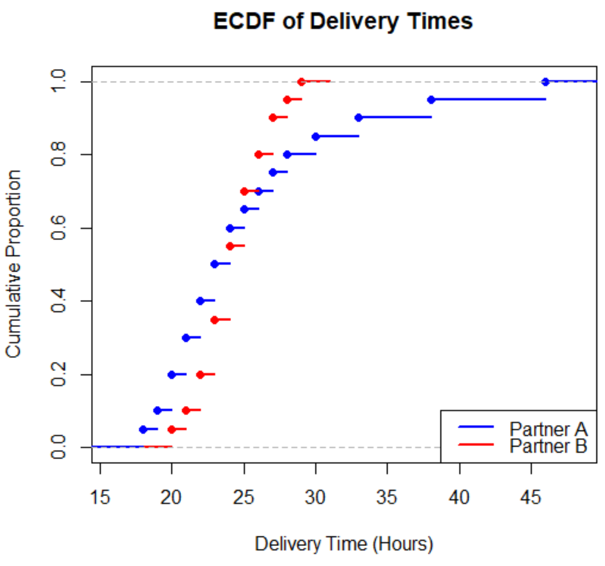
Question 1

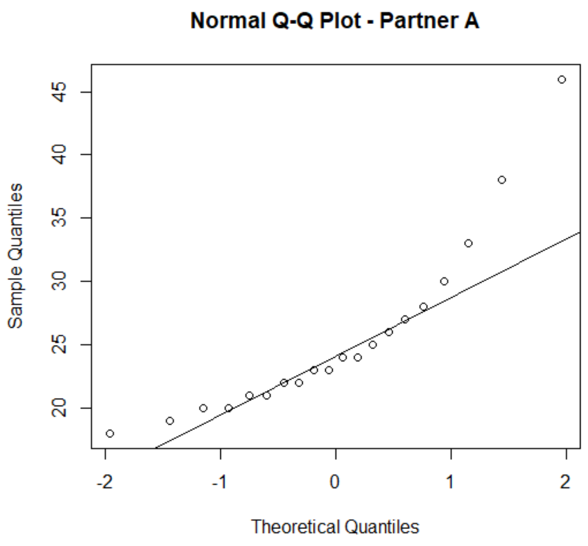
Task 1: Overlaid ECDF Plot

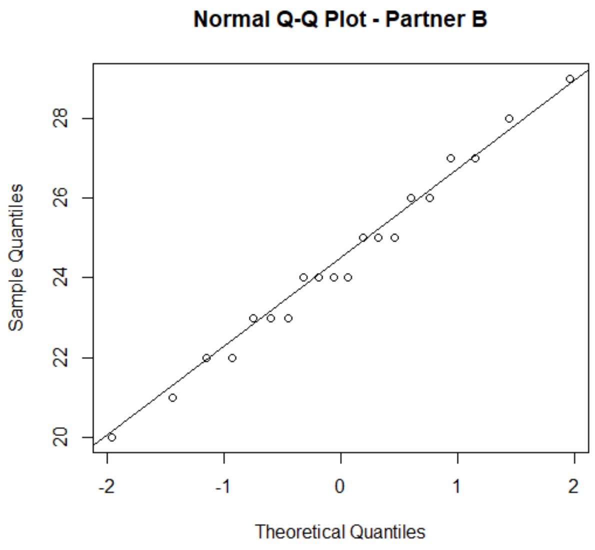
```
plot(
  ecdf(partner_a),
  col = "blue",
  lwd = 2,
  main = "ECDF of Delivery Times",
  xlab = "Delivery Time (Hours)",
  ylab = "Cumulative Proportion"
)
lines(
  ecdf(partner_b),
  col = "red",
  lwd = 2
)
legend(
  "bottomright",
  legend = c("Partner A", "Partner B"),
  col = c("blue", "red"),
  lwd = 2
)
```



Task 2: Normal Q-Q Plot for Each Partner

```
qqnorm(partner_a, main = "Normal Q-Q Plot - Partner A")

qqline(partner_a)

qqnorm(partner_b, main = "Normal Q-Q Plot - Partner B")

qqline(partner_b)
```





Task 3: Find Where ECDFs Diverge Most

```
thresholds <- sort(unique(c(partner_a, partner_b)))

ecdf_a <- ecdf(partner_a)

ecdf_b <- ecdf(partner_b)

difference <- abs(ecdf_a(thresholds) - ecdf_b(thresholds))

max_difference <- max(difference)

threshold <- thresholds[which.max(difference)]

cat("Maximum ECDF Difference:", max_difference, "\n")

cat("Delivery Time Threshold:", threshold, "hours\n")
```

Maximum ECDF Difference: 0.2
Delivery Time Threshold: 22 hours

Task 4: Check Normality Using Q-Q Plots

Partner A

Partner A is **not close to a normal distribution**. The higher values, especially **38 and 46 hours**, will cause the upper-end points to move away from the Q-Q reference line.

Interpretation: Partner A shows **right-skewness and possible high-value outliers**.

Partner B

Partner B is **closer to normal** than Partner A because its values are more concentrated between **20 and 29 hours**.

However, you should use the Q-Q plot to check how closely the points follow the line.

Conclusion: Partner B is more consistent and closer to normal, while Partner A has clear upper-tail deviation.

Task 5: SLA Recommendation

Recommended: Empirical Percentiles

Use empirical percentiles rather than **Mean ± Standard Deviation**.

Justification:

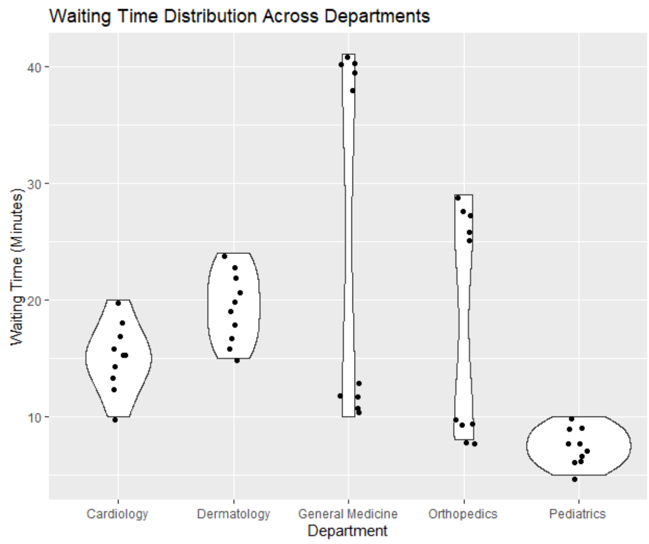
- Partner A has skewness and unusually high delivery times.
- Mean and standard deviation can be strongly affected by extreme values.
- Percentiles directly represent actual delivery performance.

Question 2

Task 1: Construct Bean Plot / Density + Jitter Visualization

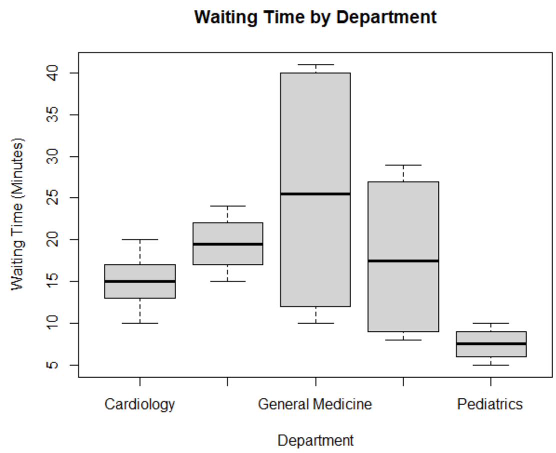
```
data <- data.frame(
  Department = rep(c(
    "Cardiology",
    "Orthopedics",
    "Pediatrics",
    "Dermatology",
    "General Medicine"
  ), each = 10),
  Waiting_Time = c(
    10,12,13,14,15,15,16,17,18,20,
```

```
    8,9,25,26,27,9,10,28,29,8,
    5,6,6,7,7,8,8,9,9,10,
    15,16,17,18,19,20,21,22,23,24,
    10,40,12,38,11,41,13,39,12,40
  )
)
print(data)
```



Task 3: Compare with a Standard Boxplot

```
boxplot(
  Waiting_Time ~ Department,
  data = data,
  main = "Waiting Time by Department",
  xlab = "Department",
  ylab = "Waiting Time (Minutes)"
)
```



Task 4: Rank Departments by Variability and Median Waiting Time

```
variability <- aggregate(
  Waiting_Time ~ Department,
  data = data,
  FUN = sd
)
variability <- variability[
  order(variability$Waiting_Time),
]
names(variability)[2] <- "Standard_Deviation"
print(variability)
```

Department Median
5 Pediatrics 7.5
1 Cardiology 15.0
4 Orthopedics 17.5
2 Dermatology 19.5
3 General Medicine 25.5

Task 5: Recommended Staffing or Scheduling Change

Answer:

General Medicine should receive additional staff during peak periods.

The distribution shows two groups: approximately **10–13 minutes** and **38–41 minutes**. This suggests that some time periods operate normally while other periods have serious delays.

Recommendation:

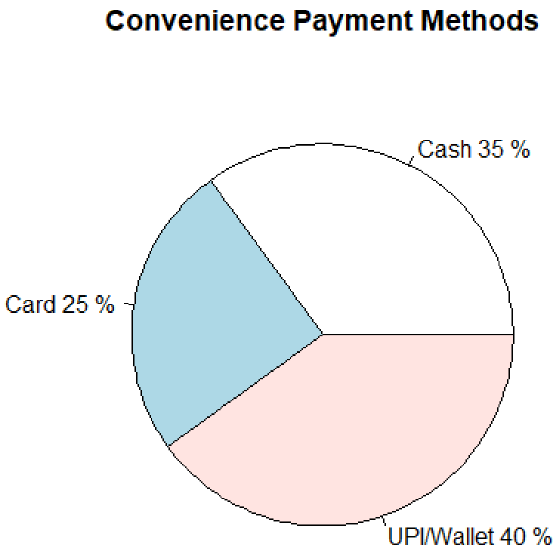
Schedule additional doctors or staff during the high-waiting-time periods to reduce the second peak and make waiting times more consistent.

Question 3 — Choosing an Honest Proportion Chart

```
# Install package if needed
```

```r
install.packages("ggplot2")

# Load package
library(ggplot2)

data <- data.frame(
  Store = c("Hypermarket", "Supermarket", "Convenience"),
  Cash = c(18, 22, 35),
  Card = c(34, 30, 25),
  UPI_Wallet = c(48, 48, 40)
)

# Convert data to long format
long_data <- data.frame(
  Store = rep(data$Store, each = 3),
  Payment = rep(c("Cash", "Card", "UPI/Wallet"), times = 3),
  Percentage = c(
    18, 34, 48,
    22, 30, 48,
    35, 25, 40
  )
)

ggplot(long_data,
       aes(x = Store, y = Percentage, fill = Payment)) +
  geom_bar(stat = "identity", position = "dodge") +
  labs(
    title = "Payment Method by Store Format",
    x = "Store Format",
    y = "Percentage (%)"
  ) +
  theme_minimal()

ggplot(long_data,
       aes(x = Store, y = Percentage, fill = Payment)) +
  geom_bar(stat = "identity", position = "fill") +
  scale_y_continuous(labels = scales::percent) +
  labs(
    title = "100% Stacked Bar Chart",
    x = "Store Format",
    y = "Percentage"
  ) +
  theme_minimal()

# Create individual datasets

hypermarket <- data.frame(
  Payment = c("Cash", "Card", "UPI/Wallet"),
  Percentage = c(18, 34, 48)
)
```

```
supermarket <- data.frame(
  Payment = c("Cash", "Card", "UPI/Wallet"),
  Percentage = c(22, 30, 48)
)

convenience <- data.frame(
  Payment = c("Cash", "Card", "UPI/Wallet"),
  Percentage = c(35, 25, 40)
)

# Pie chart - Hypermarket
pie(
  hypermarket$Percentage,
  labels = paste(hypermarket$Payment,
                 hypermarket$Percentage, "%"),
  main = "Hypermarket Payment Methods"
)

# Pie chart - Supermarket
pie(
  supermarket$Percentage,
  labels = paste(supermarket$Payment,
                 supermarket$Percentage, "%"),
  main = "Supermarket Payment Methods"
)

# Pie chart - Convenience
pie(
  convenience$Percentage,
  labels = paste(convenience$Payment,
                 convenience$Percentage, "%"),
  main = "Convenience Payment Methods"
)

upi_values <- data$UPI_Wallet

highest <- data$Store[which.max(upi_values)]

cat("Store format with highest UPI/Wallet share:",
    highest, "\n")

cat("UPI/Wallet share:",
    max(upi_values), "%\n")
cat("\nCOMPARISON:\n")

cat("Grouped Bar Chart:\n")
cat("Easiest for accurately comparing UPI/Wallet percentages.\n\n")

cat("100% Stacked Bar Chart:\n"
```

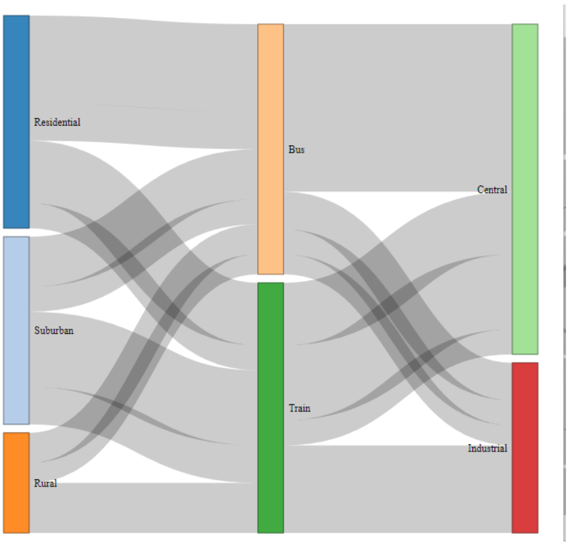


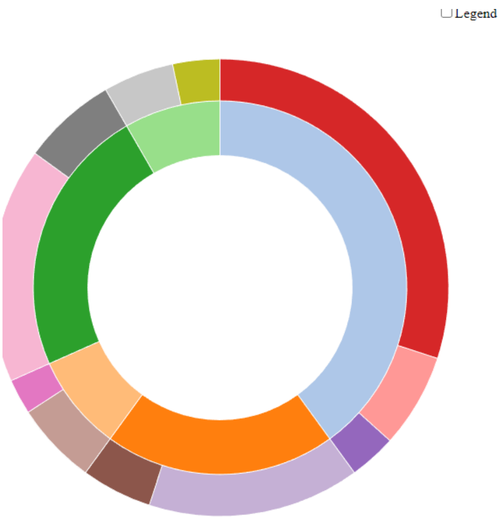
Question 4 — Nested Proportions in a Household Budget

```
# Install packages only if needed
install.packages("treemap")
install.packages("sunburstR")
# Load packages
library(treemap)
library(sunburstR)
data <- data.frame(
  Category = c(
    "Housing", "Housing", "Housing",
    "Food", "Food",
    "Transport", "Transport",
    "Savings", "Savings",
    "Discretionary", "Discretionary"
  ),
  Subcategory = c(
    "Rent/EMI", "Utilities", "Maintenance",
    "Groceries", "Dining Out",
    "Fuel", "Public Transit",
```

```
    "Investments", "Emergency Fund",
    "Entertainment", "Shopping"
  ),
  Amount = c(
    18000, 4000, 2000,
    9000, 3000,
    3500, 1500,
    10000, 4000,
    3000, 2000
  )
)
print(data)
treemap(
  data,
  index = c("Category", "Subcategory"),
  vSize = "Amount",
  title = "Household Budget Treemap"
)
sunburst_data <- data.frame(
  sequence = paste(
    data$Category,
    data$Subcategory,
    sep = "-"
  ),
  size = data$Amount
)
sunburst(sunburst_data)
total_budget <- 60000
housing_amount <- sum(
  data$Amount[data$Category == "Housing"]
```

```
)
# Largest Housing subcategory
housing_data <- data[data$Category == "Housing", ]

largest_housing <- housing_data[
  which.max(housing_data$Amount),
]
# Percentages
housing_percentage <-
  (housing_amount / total_budget) * 100

largest_subcategory_percentage <-
  (largest_housing$Amount / total_budget) * 100
cat("Housing Amount: ₹", housing_amount, "\n")
cat("Housing Percentage:", housing_percentage, "%\n")
cat("Largest Housing Subcategory:",
    largest_housing$Subcategory, "\n"
cat("Largest Subcategory Amount: ₹",
    largest_housing$Amount, "\n")
cat("Largest Subcategory Percentage:",
    largest_subcategory_percentage, "%\n")
```





Question 5 — Flow Analysis using Sankey Diagram and Parallel Sets

```
# Install packages only if needed
install.packages("networkD3")
install.packages("ggplot2")
install.packages("ggforce")
# Load packages
library(networkD3)
library(ggplot2)
library(ggforce)
data <- data.frame(
  Course = c(
    "Data Science", "Data Science",
    "Data Science", "Data Science",
    "Web Development", "Web Development",
    "Web Development", "Web Development",
    "Digital Marketing", "Digital Marketing",
    "Digital Marketing", "Digital Marketing"
  ),
  Device = c(
    "Mobile", "Mobile",
    "Laptop", "Laptop",
    "Mobile", "Mobile",
    "Laptop", "Laptop",
    "Mobile", "Mobile",
    "Laptop", "Laptop"
  ),
  Students = c(
    20, 25,
    45, 10,
```

```
    15, 20,
    35, 10,
    10, 15,
    20, 25
  )
)
print(data)
nodes <- data.frame(
  name = c(
    "Data Science",
    "Web Development",
    "Digital Marketing",
    "Mobile",
    "Laptop",
    "Completed",
    "Dropped"
  )
)
links <- rbind(links1, links2)
sankeyNetwork(
  Links = links,
  Nodes = nodes,
  Source = "source",
  Target = "target",
  Value = "value",
  NodeID = "name",
  fontSize = 12,
  nodeWidth = 30
)
parallel_data <- gather_set_data(
```

```
  data,
  c("Course", "Device", "Status"),
  id_name = "FlowID"
)
ggplot(
  parallel_data,
  aes(
    x = x,
    id = FlowID,
    split = y,
    value = Students
  )
) +
  geom_parallel_sets(
    aes(fill = Course),
    alpha = 0.7
  ) +
  geom_parallel_sets_axes() +
names(completion_rate) <- c(
  "Course",
  "Total",
  "Completed"
)
completion_rate$Completion_Percentage <-
  (completion_rate$Completed /
   completion_rate$Total) * 100
print(completion_rate)
weakest <- completion_rate[
  which.min(completion_rate$Completion_Percentage),
]
```

```
cat(
  "\nWeakest Course:",
  weakest$Course,
  "\nCompletion Rate:",
  weakest$Completion_Percentage,
  "%\n"
```